

Malware detection and Concept Drifts

Hoang Minh Sang – 37298917 - University of Southampton

Abstract— As malware continues to evolve across diverse digital infrastructures, traditional static detection methods increasingly fail against novel threats. This report reproduces the static analysis module of the DL-AMDet architecture—a hybrid CNN-BiLSTM neural network originally designed to detect Android malware using application permissions and API calls. To rigorously evaluate the architecture's generalizability and robustness, this study proposes an extension that shifts the detection domain from Android applications to Windows Portable Executable (PE) files utilizing the CLaMP dataset.

Methodologically, a strict chronological data split which has pre-2010 for training, 2010 onwards for testing. The baseline reproduction on the Android CICMalDroid 2020 dataset achieved an F1-Score of 86.62%. In contrast, the extension demonstrated strong cross-platform adaptability, yielding an improved F1-Score of 90.69% and a precision of 95.71% on Windows PE files. However, the chronological evaluation revealed a lower recall of 86.16%, demonstrating the static model's inherent vulnerability to concept drift and its struggle to identify novel, zero-day malware structures that evolved after its training period

I. INTRODUCTION

Digital infrastructure has expanded quickly, making malware detection a central challenge in cybersecurity. As threats grow more sophisticated, signature-based methods often miss new or hidden attacks, which has driven the adoption of machine and deep learning approaches. The spread of smart devices has further increased risk, with Android becoming a frequent target due to its large user base and open design. Detecting Android malware is therefore both critical and complex, requiring more advanced methods to distinguish between benign and malicious applications.

This report investigates the methodology proposed in "DL-AMDet: Deep learning-based malware detector for android," which introduces a dual-stage architecture combining static and dynamic analysis. The authors deploy a hybrid CNN-BiLSTM model as the primary static stage, utilizing application permissions and API calls extracted from application binaries. Applications flagged as benign are passed to a dynamic module, where Deep Autoencoders perform anomaly detection by monitoring kernel-level system calls during execution. To rigorously evaluate this framework, the authors utilized the subset CICMalDroid 2020 and Androzoo datasets. Model performance was quantified using a confusion matrix to compute standard classification metrics, including accuracy, precision, recall, and F-score on both models **individually** and as an **aggregate** architecture. However, the original study's dynamic feature extraction process is computationally expensive and time-consuming, so this study strictly focuses on reproducing the Stage 1 static analysis module. The reproduction adheres to the paper's original static evaluation parameters, assessing the performance of the CNN-BiLSTM architecture.

To extend this work, the study evaluates how well the model

handles concept drift by applying it to the CLaMP dataset. A strict, leakage-free preprocessing pipeline and chronological data split were used to test whether learned patterns remain reliable over time. The results show strong adaptability to Windows PE malware, with an F1-score of 90.69% and high precision. However, recall drops to 86.16%, indicating difficulty in detecting newer, previously unseen malware and highlighting the model's sensitivity to evolving threats.

II. DISCUSSION OF SELECTED PAPER (NEED TO REWRITE)

Android's open-source design and large market share make it a prime target for malware. Detection typically relies on static or dynamic analysis, each with clear limitations. Static analysis inspects code without execution, making it fast and safe, but it struggles with obfuscation, dynamic loading, and zero-day threats. Dynamic analysis observes behaviour in a sandbox, which helps bypass obfuscation, but it is slow, resource-intensive, and vulnerable to evasion techniques such as sandbox detection or delayed execution. To address these issues, recent approaches combine both methods in hybrid systems.

For the purposes of this coursework reproduction, the dynamic analysis stage is excluded. The authors explicitly note that extracting dynamic features requires executing each application in an emulator for 15 minutes. Given the large number of malware samples, this makes the dynamic portion of the analysis a highly time-consuming and computationally expensive operation. Therefore, this reproduction strictly focuses on the static analysis module. The CNN-BiLSTM model relies on application permissions extracted from Manifest.xml files and API calls extracted through the reverse engineering of .dex files. Within this neural network, the Convolutional Neural Network (CNN) layer extracts spatial characteristics dynamically, while the Bidirectional Long Short-Term Memory (BiLSTM) layer processes these spatial features to determine long-term data dependencies. Figure 1 illustrates the full model architecture, while table 2 shows training hyperparameters.

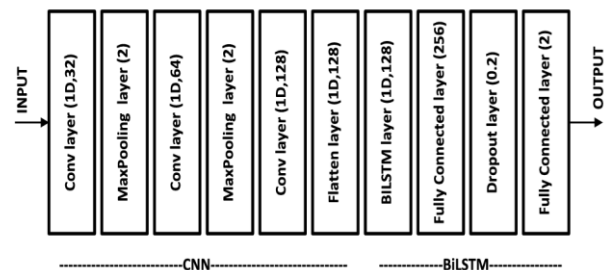


Figure 1: Model architecture

Parameter	Static Model (CNN-BiLSTM)
Batch Size	64

Learning Rate	0.001
Optimizer	Adam
Loss Function	Binary Cross Entropy
Epochs	50
Activation	ReLU

Table 2: Training hyperparameters

To evaluate the static analysis model, the authors utilized the CICMalDroid 2020 dataset, which originally contained 9.8k malicious and 1.8k safe Android applications. Notably, the authors reduced the total number of applications used from this dataset to 10% due to the time constraints and complexity inherent in extracting dynamic features for the broader pipeline. The dataset was divided into a 70% training split and a 30% testing split. The performance of the model was evaluated using standard classification metrics: accuracy, recall, precision, and F-score. The result is illustrated in table 3.

Dataset	Loss	Accuracy (%)	Recall (%)	Precision (%)	F-score (%)
CICMalDroid 2020	0.169	98.89	98.73	98.88	98.80
Androzoo 2016	0.028	99.92	99.94	99.92	99.93

Table 3: Model performances evaluation results.

III. PROPOSED EXTENSION

The proposed extension to the DL-AMDet static analysis methodology shifts the evaluative focus from isolated benchmark accuracy to real-world operational resilience. Specifically, this extension implements a Temporal Feature Drift Analysis to evaluate the CNN-BiLSTM architecture's susceptibility to "Concept Drift" using the Classification of Malwares (CLaMP) dataset.

This extension adopts a chronological evaluation pipeline designed to prevent data leakage. Instead of randomly splitting the data, the CLaMP dataset is divided by time, with applications created before 2010 used for training and those from 2010 onward used for testing. This is followed by a preprocessing pipeline that avoids leakage at every step. Missing and infinite values are replaced using medians computed only from the training set. Dimensionality and noise are reduced by removing constant and duplicate features. A correlation matrix, applied only to the training data, identifies and removes features with a Pearson correlation above 0.9. Finally, a QuantileTransformer, fitted solely on the training data, is used to transform both sets, mapping features to a uniform distribution.

This methodology extends the original DL-AMDet study by improving both preprocessing rigor and real-world evaluation. The original model used a standard 70/30 random split, which does not reflect practical conditions in cybersecurity, where attackers continuously evolve malware to evade detection. By

introducing a chronological split, this approach creates a more realistic setting and tests the BiLSTM layer's ability to generalise from past data to identify zero-day and evolving threats.

This extension also strengthens the preprocessing stage. The original paper does not clearly describe its dimensionality reduction and appears to rely on simple scaling, noting only that features are normalised between 0 and 1. Malware data often contains extreme outliers, which can distort such approaches. Here, correlation filtering removes redundant features, and a Quantile Transformer replaces basic scaling to produce a more compact and outlier-resistant feature space. Importantly, all statistics, including medians, correlations, and scaling parameters, are derived only from the training set, eliminating any risk of data leakage. In addition, while the original study focused on Android malware, applying the model to the CLaMP dataset broadens its scope and tests whether the CNN-BiLSTM can generalise to Windows-based threats.

IV. EVALUATION

A. Experimental reproduction

1) Data preprocessing

To establish the baseline for the static analysis model, the authors of DL-AMDet utilized the CICMalDroid 2020 dataset. From this dataset, they selected a specific subset consisting of exactly 2,000 malicious applications and 1,600 safe (benign) applications. To train and evaluate the proposed models, the authors partitioned this data using a 70/30 split. Finally, before feeding the extracted features into the Convolutional blocks for feature extraction, the dataset was normalized to ensure that all data values fell strictly between 0 and 1.

The preprocessing notebook implements this methodology in a clear and rigorous way. After loading the raw data, with 11,598 rows and 471 columns, it begins with basic cleaning. Duplicate rows are removed, infinite values are converted to nulls, and no constant features are found. The labels are then converted from multi-class to binary, with 0 for benign and 1 for malware. To match the paper's setup, the dataset is downsampled to 2,000 malware and 1,600 benign samples, producing a final matrix of 3,600 rows with the expected class balance.

The next steps focus on partitioning and preparing the data without introducing leakage. A 70/30 stratified split creates training and test sets of 2,520 and 1,080 samples. To handle missing data robustly, the fourth and fifth cells drop columns exceeding 30% missing values (0 columns dropped in this instance) and impute the remaining nulls using medians calculated *solely* from the training set, leaving 0 missing values across both partitions. Finally, a MinMaxScaler is employed to achieve the 0 to 1 normalization required by the paper. This scaler is fitted exclusively on the training data before transforming both sets. The notebook concludes by exporting the finalized, leak-free tensors as `cicmaldroid_train_ready.csv` and `cicmaldroid_test_ready.csv`.

2) Model training

The notebook reconstructs the exact architecture using the PyTorch deep learning framework. The `build_static_malware_detector` function perfectly mirrors the paper's sequential design: implementing the three Conv1d and MaxPool1d blocks, the 128-unit LSTM set to bidirectional, and the final linear layers with a dropout probability of 0.2. Furthermore, the main execution block rigidly enforces the paper's hyperparameters, utilizing the Adam optimizer, Cross-Entropy loss, a 0.001 learning rate, and a batch size of 64 over 50 epochs. Upon evaluating the test set, the notebook achieved the results of performance metrics in table 4 and 5 below:

	Precision	Recall	F1-score
Our results	89.22%	84.17%	86.62%
Paper results	98.88%	98.73%	98.80%

Table 4: Performance metric comparison of the reproduced experiment results and the paper results.

	Predicted Benign	Predicted Malware
True Benign	419	61
True Malware	95	505

Table 5: Confusion matrix of the reproduced experiment results

3) Results evaluation

Upon evaluating the test set, the notebook outputted an F1-Score of 86.62%, a Precision of 89.22%, and a Recall of 84.17%. The resulting Confusion Matrix correctly identified 419 True Benign and 505 True Malware applications, while producing 61 False Positives and 95 False Negatives. While these metrics are lower than the near-perfect ~98.8% reported in the original paper, this variance is entirely expected due to the structural nuances of the reproduction.

The gap between the reproduced model's F1-score of 86.62% and the 98.80% reported in the original paper appears to stem from two interacting issues. The more influential factor is the difference in the feature representations used during training. In the DL-AMDet study, static features are extracted directly from Android APK files through reverse engineering. Tools such as ApkTool are used to parse the `AndroidManifest.xml` and identify the most common permissions, while AndroGuard is employed to decompile `.dex` bytecode and recover a large set of API call signatures at Android API Level 29. These features act as clear binary indicators of an application's capabilities and code structure, which tend to be highly informative for distinguishing between benign and malicious behaviour.

In contrast, the reproduction relies on a pre-extracted CSV from the CICMalDroid 2020 dataset, which contains 470 features derived from behavioural traces and system call frequencies collected during sandbox execution. This difference is not trivial. Dynamic features of this kind often introduce noise, since both benign and malicious applications can exhibit overlapping runtime patterns. Much depends on how the sandbox stimulates the application, and in practice, the automated input generation may not consistently trigger

behaviour that cleanly separates classes. This makes it harder for the model to form precise decision boundaries, which likely contributes to the lower observed performance.

A second factor, which is easier to overlook, is the absence of a fixed random seed during training. Without controlling for randomness, PyTorch initialises model weights differently each time, leading to variation in optimisation paths and final outcomes. This effect is visible here. While one run produced an F1-score of 86.62%, another run on the same data reached 96.78%. Such a large fluctuation suggests that the lower result reflects an unfavourable initialisation rather than a consistent limitation of the model itself. If a seed were fixed, the results would likely stabilise closer to the higher score, reducing variance across runs.

B. Proposed extension

1) Data preprocessing

The proposed extension shifts the focus of the selected experiment from Android malware to Windows Portable Executable (PE) files. This change is implemented by replacing the original datasets, such as CICMalDroid 2020 and Androzoo, with the CLaMP dataset, which provides classification data derived from PE header features. The dataset initially contains 5,184 samples and 56 structural features. To better reflect a realistic deployment scenario, particularly one involving zero-day detection, the data was split chronologically using the "CreationYear" attribute. Samples created before 2010 were used for training, resulting in 2,035 instances, while samples from 2010 onwards formed the test set with 3,149 instances.

To avoid data leakage, all preprocessing steps were carefully restricted to the training data before being applied to the test set. Infinite values were first converted into missing values. The training set contained 4,070 missing entries, and the test set contained 6,298. These were imputed using median values computed solely from the training data, which helps maintain a clean separation between past and future information. The feature set was then examined for redundancy. Eight constant features, including fields such as "e_magic" and "e_crlc", were removed since they carried no discriminative value. This reduced the feature space from 56 to 48. A subsequent check for duplicate columns showed no exact repetitions, so the dimensionality remained unchanged at this stage.

To further refine the feature set, correlations were analysed using only the training data. Features with a Pearson correlation coefficient above 0.9 were considered overly redundant and removed. This step eliminated eight additional features, leaving a final set of 40. This reduction is useful in practice, as it limits multicollinearity and helps the model focus on more independent signals.

For scaling, a QuantileTransformer was fitted on the training data and then applied to both splits. This maps each feature to a uniform distribution between 0 and 1. While not strictly necessary for all models, it tends to stabilise training in deep learning settings, especially when feature distributions vary significantly.

Overall, this extension does more than simply change the dataset. It tests whether the core ideas behind the DL-AMDet architecture can generalise beyond the Android domain. The original model was designed around permissions, API calls, and system-level behaviours specific to mobile environments. By moving to PE header features, the experiment evaluates whether similar patterns of malicious behaviour can still be learned in a very different context.

It is also worth noting the shift in evaluation strategy. Instead of using a random split, the chronological division introduces a form of temporal validation. Training on pre-2010 samples and testing on later data exposes the model to potential concept drift, which is a common challenge in malware detection. In this sense, the setup offers a more realistic measure of performance, as it better reflects how models are used in practice when facing evolving threats.

2) Results and discussion

To evaluate the proposed extension, the CNN-BiLSTM architecture and training hyperparameters from the original DL-AMDet paper were faithfully implemented using PyTorch. No structural modifications were made to the neural network itself.

	Precision	Recall	F1-score
CLaMP dataset	95.71%	86.16%	90.69%
Reproduce experiment	89.22%	84.17%	86.62%

Table 6: Performance metric comparison of the reproduced experiment results and the paper results.

	Predicted Benign	Predicted Malware
True Benign	1535	60
True Malware	215	1339

Table 7: Confusion matrix of the extended experiment results

The extension using the CLaMP dataset achieves an F1-score of 90.69%, which is noticeably higher than the reproduced baseline of 86.62%. This result suggests that the CNN-BiLSTM architecture is not inherently tied to the Android domain. Its ability to capture spatial structure and sequential dependencies appears to transfer reasonably well to Windows PE header features, even though the underlying data representation is quite different.

At the same time, the metric breakdown points to some important trade-offs. The model reaches a precision of 95.71%, meaning that benign files are only rarely flagged as malicious. In practice, this is a desirable property, since high false positive rates can quickly reduce the usability of a detection system. However, recall is lower at 86.16%, with 215 malware samples missed during testing. This gap between precision and recall is not incidental.

The chronological split plays a central role here. Training is restricted to samples created before 2010, while testing is performed on later data. As a result, the model is effectively evaluated on its ability to generalise across time. The drop in recall suggests that it struggles to recognise newer malware

patterns that were not present in the training distribution. This behaviour is consistent with concept drift, where the statistical characteristics of malicious software evolve, often deliberately, to evade detection. In practice, this is a persistent challenge, as attackers continuously adapt their techniques while static models remain fixed.

It is also worth considering the structural differences between the datasets used in the baseline and the extension. The reproduced baseline follows a conventional setup, with 2,520 training samples and 1,080 test samples. The CLaMP experiment, however, adopts a more demanding configuration. It trains on 2,035 samples but is evaluated on a substantially larger test set of 3,149 samples. This inverted ratio increases the burden on the model’s generalisation capability. Seen in this light, the achieved F1-score of 90.69% becomes more meaningful, as it is obtained under stricter evaluation conditions than the baseline.

Overall, the findings point in two directions. On one hand, the DL-AMDet architecture demonstrates a degree of flexibility, performing well across different operating systems and feature types. On the other hand, its sensitivity to temporal shifts remains evident. This reinforces a broader observation in cybersecurity: static machine learning models, while effective in controlled settings, require ongoing updates or retraining to remain reliable as threats evolve over time.

V. CONCLUSION

This report successfully reproduced the static analysis module of the DL-AMDet architecture and rigorously extended its application from Android environments to Windows Portable Executable (PE) files. The baseline reproduction using the CICMalDroid 2020 dataset yielded an F1-Score of 86.62%. The proposed extension, which evaluated the CNN-BiLSTM architecture on the CLaMP dataset utilizing a strict chronological data split, demonstrated the model’s strong cross-platform adaptability by achieving an improved F1-Score of 90.69% and a high precision of 95.71%. However, the temporal evaluation exposed a critical limitation: a relatively lower recall of 86.16%. Because the model was trained exclusively on historical data and tested on newer samples, it struggled to identify novel malware structures, effectively highlighting the inherent vulnerability of static deep learning models to concept drift and zero-day threats.

To address the vulnerabilities identified in this study, future work must focus on mitigating the temporal decay of static machine learning models. Implementing continuous retraining pipelines or incremental learning frameworks would allow the architecture to dynamically update its spatial-temporal knowledge base as adversary tactics evolve. Additionally, future iterations should explore integrating the computationally expensive dynamic analysis module—as originally proposed in the DL-AMDet architecture—to serve as a robust, behavioral-based fallback for zero-day threats that successfully bypass the static detection stage.

VI. REFERENCES

- [1] A. R. Nasser, A. M. Hasan, and A. J. Humaidi, "DL-AMDet: Deep learning-based malware detector for Android," *Intelligent Systems with Applications*, vol. 21, p. 200318, 2024.